CASE 01: Progressive Electricity Billing

1. Problem Statement

Electricity charges are usually calculated in slabs, where the rate per unit increases as consumption increases. Calculating such bills by hand is slow and error-prone. This project develops a Python program that takes the number of units consumed, applies slab-wise rates, adds a fixed charge, and returns the final bill. Invalid (negative) consumption must be reported as invalid.

2. Objectives

1. To develop a Python function that calculates an electricity bill from units consumed.
2. To apply different rates for different consumption slabs.
3. To add a fixed service charge to every valid bill.
4. To reject invalid (negative) unit readings.
5. To verify the function using five test cases.

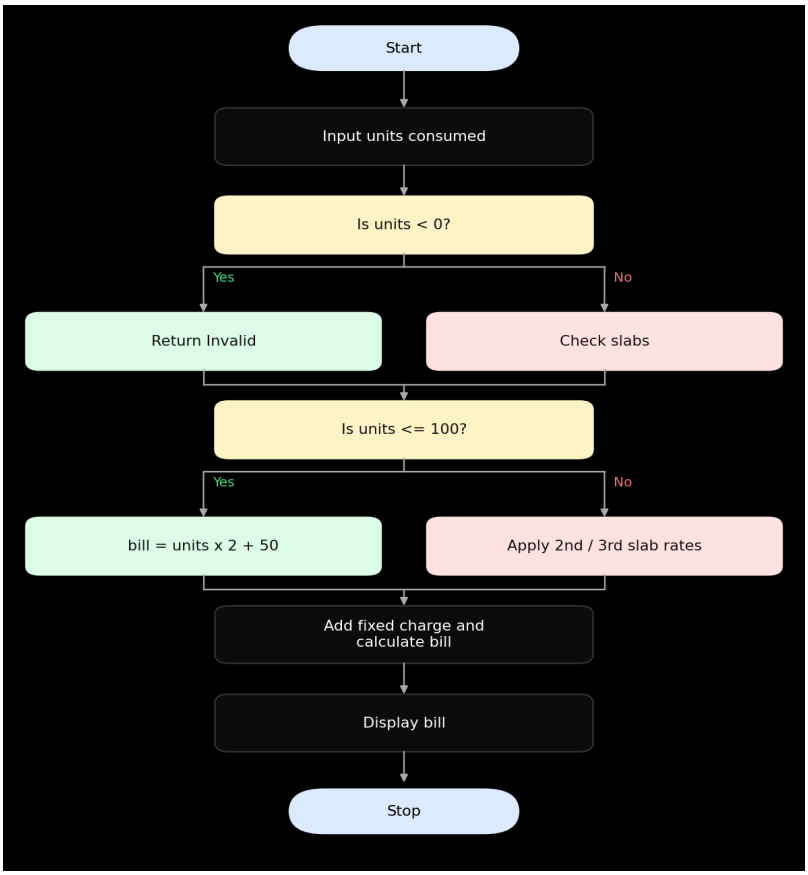
3. Assumptions

1. The first 100 units are charged at Rs. 2 per unit.
2. Units from 101 to 200 are charged at Rs. 3 per unit.
3. Units above 200 are charged at Rs. 5 per unit.
4. A fixed charge of Rs. 50 is added to every valid bill.
5. Units consumed are entered as whole numbers.
6. A negative number of units is treated as invalid input.
7. These are sample rates for learning, not official tariffs.

4. Algorithm

1. Start the program.
2. Read the units consumed.
3. If units are less than 0, return "Invalid".
4. If units are 100 or fewer, bill = units x 2 + 50.
5. Else if units are 200 or fewer, bill = 100 x 2 + (units - 100) x 3 + 50.
6. Otherwise, bill = 100 x 2 + 100 x 3 + (units - 200) x 5 + 50.
7. Display the bill amount.
8. Repeat for every test case.
9. Stop the program.

5. Flowchart



6. Pseudocode

```
START

INPUT units

IF units < 0
    RETURN "Invalid"
ELSE IF units <= 100
    bill = units * 2 + 50
ELSE IF units <= 200
    bill = 100 * 2 + (units - 100) * 3 + 50
ELSE
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5 + 50
END IF

DISPLAY bill

STOP
```

7. Python Source Code / Program

```
def bill(units):
    if units < 0:
        return "Invalid"
    elif units <= 100:
        return units * 2 + 50
    elif units <= 200:
        return 100 * 2 + (units - 100) * 3 + 50
    else:
        return 100 * 2 + 100 * 3 + (units - 200) * 5 + 50

test_cases = [50, 80, 100, 150, 250]

for i in range(5):
    print("Test Case", i+1, ":", bill(test_cases[i]))
```

8. Test Cases

Test case | Units consumed | Expected bill

1 | 50 | Rs. 150

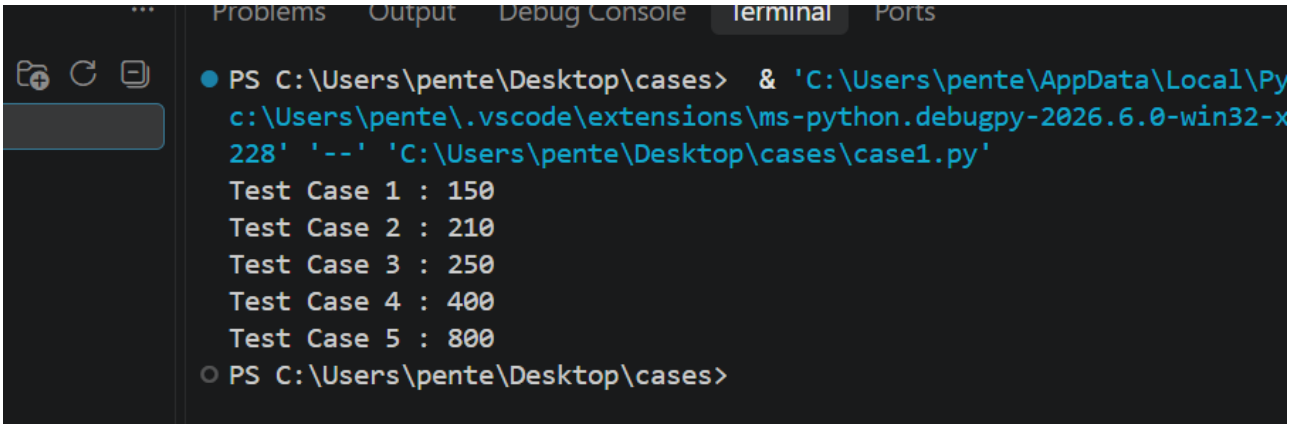
2 | 80 | Rs. 210

3 | 100 | Rs. 250

4 | 150 | Rs. 400

5 | 250 | Rs. 800

9. Output Screenshot



10. Conclusion

This program calculates an electricity bill using slab-wise rates and a fixed charge, and rejects invalid readings. The five test cases confirm that every slab boundary produces the correct amount.

CASE 02: ATM Transaction Simulation

1. Problem Statement

An ATM must process a series of deposits and withdrawals while protecting the account from invalid operations. This project develops a Python program that starts with an opening balance, processes a list of transactions, ignores invalid amounts, and rejects withdrawals that exceed the per-transaction limit or the available balance.

2. Objectives

1. To simulate deposit and withdrawal transactions on an account.
2. To ignore transactions with zero or negative amounts.
3. To enforce a per-transaction withdrawal limit.
4. To prevent withdrawals larger than the available balance.
5. To return the final account balance after all transactions.

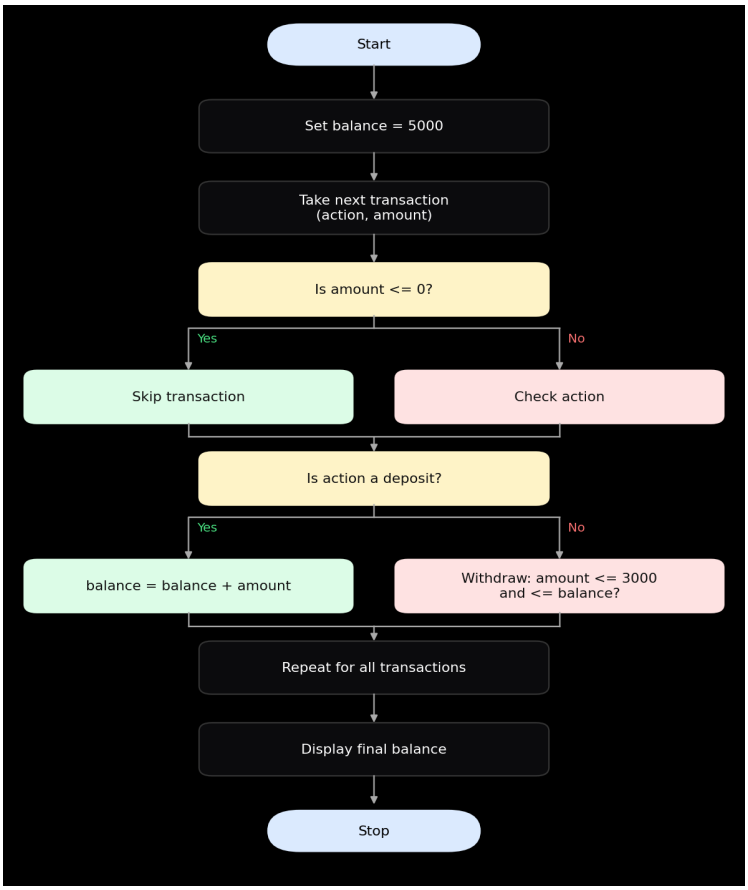
3. Assumptions

1. The opening balance is Rs. 5000.
2. The maximum amount for one withdrawal is Rs. 3000.
3. Each transaction is an (action, amount) pair, where the action is "deposit" or "withdraw".
4. Transactions with an amount of 0 or less are skipped.
5. A rejected withdrawal leaves the balance unchanged.
6. Transactions are processed in the order given.

4. Algorithm

1. Start the program.
2. Set the balance to 5000.
3. Take the next transaction (action, amount).
4. If the amount is 0 or less, skip the transaction.
5. If the action is deposit, add the amount to the balance.
6. If the action is withdraw, check that the amount is at most 3000 and at most the balance.
7. If both checks pass, subtract the amount from the balance; otherwise ignore it.
8. Repeat until all transactions are processed.
9. Return and display the final balance.
10. Stop the program.

5. Flowchart



6. Pseudocode

```
START

SET balance = 5000

FOR each (action, amount) in transactions:
    IF amount <= 0
        CONTINUE
    ELSE IF action = "deposit"
        balance = balance + amount
    ELSE IF action = "withdraw"
        IF amount <= 3000 AND amount <= balance
            balance = balance - amount

RETURN balance

STOP
```

7. Python Source Code / Program

```
def atm(transactions):
    balance = 5000
    for action, amount in transactions:
        if amount <= 0:
            continue
        elif action == "deposit":
            balance += amount
        elif action == "withdraw":
            if amount <= 3000 and amount <= balance:
                balance -= amount
    return balance
test_cases = [
    [("deposit", 1000)],
    [("withdraw", 2000)],
    [("deposit", 1000), ("withdraw", 500)],
    [("withdraw", 4000)],
    [("withdraw", 6000)]
]
for i in range(5):
    print("Test Case", i+1, ":", atm(test_cases[i]))
```

8. Test Cases

Test case | Transactions | Expected balance

1 | Deposit 1000 | 6000

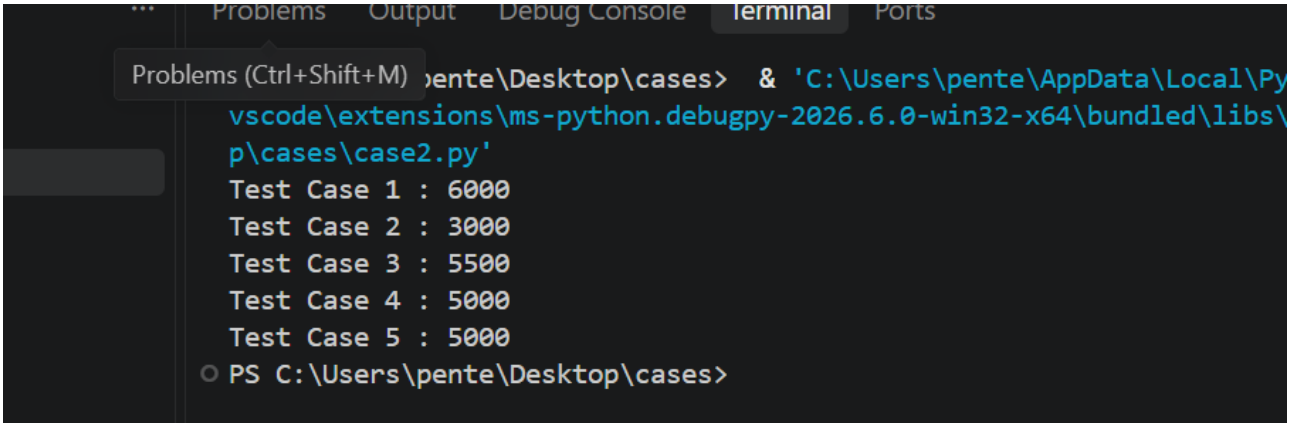
2 | Withdraw 2000 | 3000

3 | Deposit 1000, withdraw 500 | 5500

4 | Withdraw 4000 (above limit) | 5000

5 | Withdraw 6000 (above balance) | 5000

9. Output Screenshot



10. Conclusion

This program simulates ATM deposits and withdrawals while enforcing the withdrawal limit and balance checks. Invalid or oversized requests are rejected without changing the balance.

CASE 03: Password Validation

1. Problem Statement

Weak passwords make accounts easy to compromise. This project develops a Python function that checks a password against a fixed set of rules (length, character categories, and a prohibited word) and reports whether it is a valid or invalid password.

2. Objectives

1. To validate passwords against predefined security rules.
2. To check the minimum length of 8 characters.
3. To confirm the presence of uppercase, lowercase, digit and special characters.
4. To reject passwords that contain the word "password" in any letter case.
5. To verify the function using five test cases.

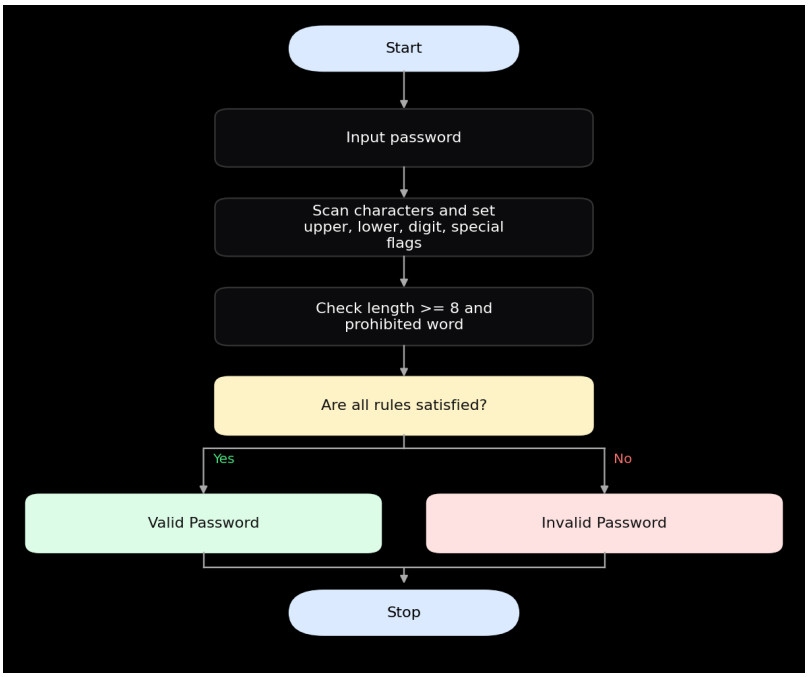
3. Assumptions

1. The minimum password length is 8 characters.
2. At least one uppercase letter (A-Z) is required.
3. At least one lowercase letter (a-z) is required.
4. At least one digit (0-9) is required.
5. Any character that is not a letter or digit counts as a special character.
6. The substring "password" is prohibited, regardless of letter case.
7. A password is valid only when every rule is satisfied.

4. Algorithm

1. Start the program.
2. Read the password.
3. Set the flags upper, lower, digit and special to False.
4. Scan each character and update the matching flag.
5. Check that the length is at least 8.
6. Check that all four flags are True.
7. Check that "password" is not present in the lower-case password.
8. If all checks pass, display "Valid Password".
9. Otherwise, display "Invalid Password".
10. Stop the program.

5. Flowchart



6. Pseudocode

```
START

INPUT password
SET upper = lower = digit = special = False

FOR each character ch in password:
    IF ch is uppercase     SET upper = True
    ELSE IF ch is lowercase SET lower = True
    ELSE IF ch is a digit   SET digit = True
    ELSE                    SET special = True

IF length >= 8 AND upper AND lower AND digit AND special
   AND "password" not in lowercase(password)
    DISPLAY "Valid Password"
ELSE
    DISPLAY "Invalid Password"

STOP
```

7. Python Source Code / Program

```
def validate(p):
    upper = lower = digit = special = False
    for ch in p:
        if ch.isupper():
            upper = True
        elif ch.islower():
            lower = True
        elif ch.isdigit():
            digit = True
        else:
            special = True
    if len(p) >= 8 and upper and lower and digit and special and "password" not in p.lower():
        return "Valid Password"
    return "Invalid Password"
test_cases = [
    "Hello@123",
    "python123",
    "HELLO@123",
    "Hi@12",
    "Password@123"
]
for i in range(5):
    print("Test Case", i+1, ":", validate(test_cases[i]))
```

8. Test Cases

Test case | Input password | Expected result

1 | Hello@123 | Valid Password

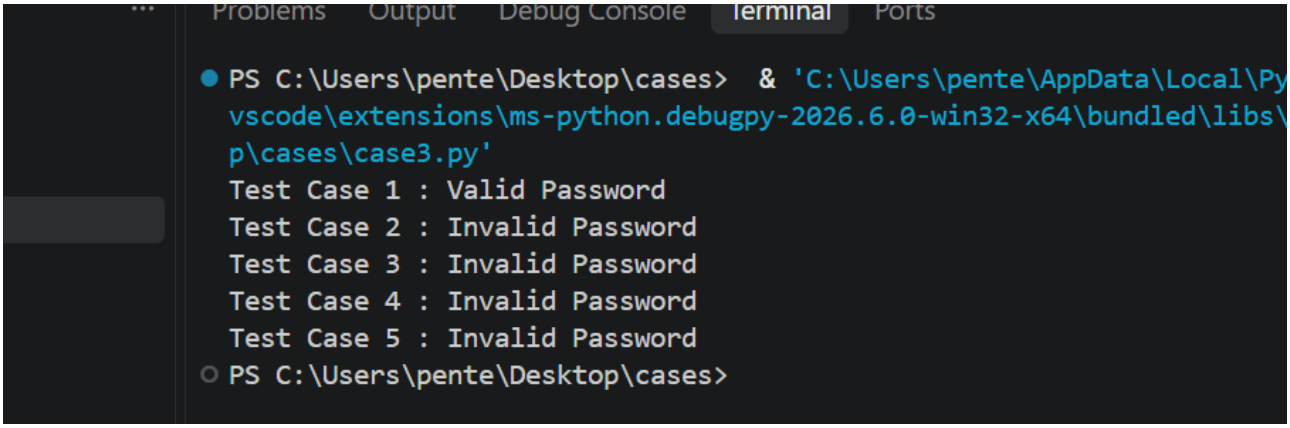
2 | python123 | Invalid - uppercase and special missing

3 | HELLO@123 | Invalid - lowercase missing

4 | Hi@12 | Invalid - shorter than 8 characters

5 | Password@123 | Invalid - contains prohibited word

9. Output Screenshot



10. Conclusion

This program validates passwords by combining length, character-category and prohibited-word checks. The test cases show that each rule can independently cause a password to be rejected.

CASE 04: Examination Answer Evaluation

1. Problem Statement

Evaluating objective answer sheets by hand is slow and inconsistent. This project develops a Python function that compares a student's answers with an answer key, counts correct, wrong and unanswered questions, and calculates a score using negative marking.

2. Objectives

1. To compare a student's answers with the answer key.
2. To count correct, wrong and unanswered questions.
3. To apply 2 marks for each correct answer and a penalty for each wrong answer.
4. To leave unanswered questions without any penalty.
5. To return the counts and final score for each answer sheet.

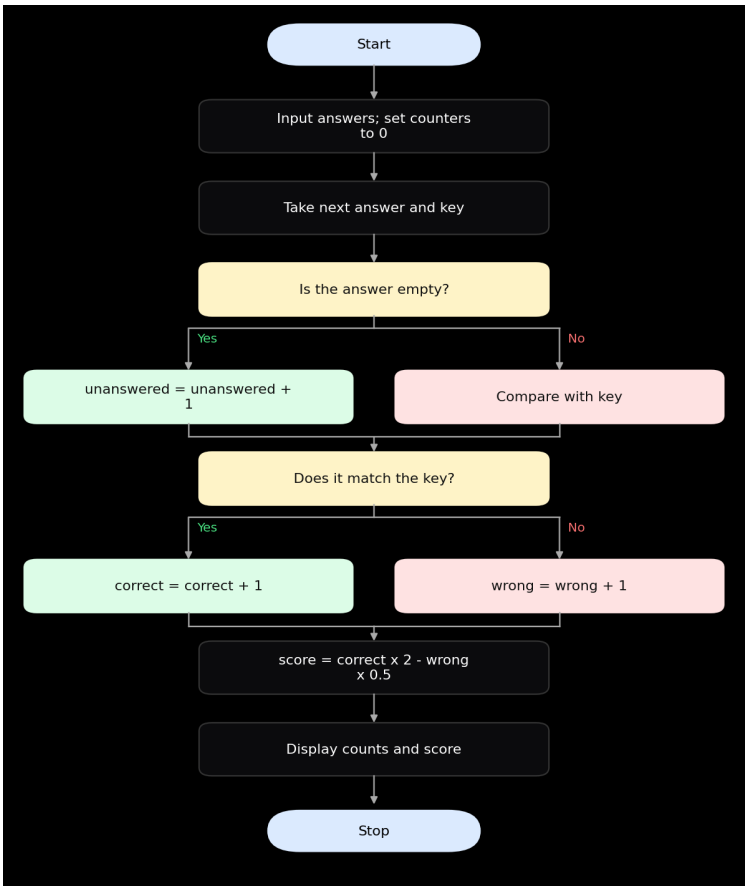
3. Assumptions

1. The answer key is [A, B, C, D, A] (five questions).
2. A correct answer earns 2 marks.
3. A wrong answer loses 0.5 marks.
4. An unanswered question (empty string) earns 0 marks and no penalty.
5. Every answer sheet contains five entries.
6. Answers are entered as capital letters.

4. Algorithm

1. Start the program.
2. Define the answer key.
3. Set correct, wrong and unanswered to 0.
4. Compare each answer with the matching key entry.
5. If the answer is empty, increase unanswered.
6. If it matches the key, increase correct; otherwise increase wrong.
7. Calculate score = correct x 2 - wrong x 0.5.
8. Return (correct, wrong, unanswered, score).
9. Stop the program.

5. Flowchart



6. Pseudocode

```
START

SET key = [A, B, C, D, A]
SET correct = wrong = unanswered = 0

FOR each answer a and key entry k:
    IF a is empty
        unanswered = unanswered + 1
    ELSE IF a = k
        correct = correct + 1
    ELSE
        wrong = wrong + 1

score = correct * 2 - wrong * 0.5

RETURN correct, wrong, unanswered, score

STOP
```

7. Python Source Code / Program

```
def evaluate(answers):
    key = ["A", "B", "C", "D", "A"]
    correct = wrong = unanswered = 0
    for a, k in zip(answers, key):
        if a == "":
            unanswered += 1
        elif a == k:
            correct += 1
        else:
            wrong += 1
    score = correct * 2 - wrong * 0.5
    return correct, wrong, unanswered, score
test_cases = [
    ["A", "B", "C", "D", "A"],
    ["A", "B", "C", "A", "B"],
    ["", "", "", "", ""],
    ["A", "C", "C", "D", "A"],
    ["B", "B", "", "D", "C"]
]
for i in range(5):
    print("Test Case", i+1, ":", evaluate(test_cases[i]))
```

8. Test Cases

Test case | Answers | Expected (correct, wrong, unanswered, score)

1 | A B C D A | (5, 0, 0, 10.0)

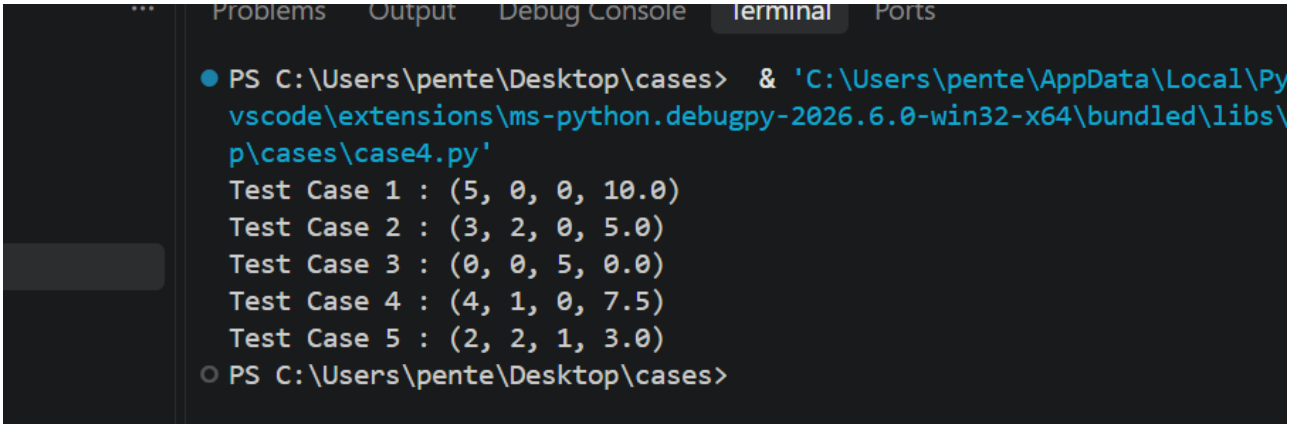
2 | A B C A B | (3, 2, 0, 5.0)

3 | all blank | (0, 0, 5, 0.0)

4 | A C C D A | (4, 1, 0, 7.5)

5 | B B blank D C | (2, 2, 1, 3.0)

9. Output Screenshot



10. Conclusion

This program evaluates answer sheets consistently, applying positive marks, negative marks and no penalty for blanks. The results match the hand-calculated scores for all five test cases.

CASE 05: Student Feedback Sentiment

1. Problem Statement

Colleges collect large amounts of written student feedback, and reading every comment is time-consuming. This project develops a simple Python program that classifies a feedback comment as Positive, Negative or Neutral by counting positive and negative keywords.

2. Objectives

1. To classify feedback text as Positive, Negative or Neutral.
2. To use predefined lists of positive and negative words.
3. To convert text to lower case so matching is not case-sensitive.
4. To calculate a sentiment score from the matched words.
5. To verify the function using five test cases.

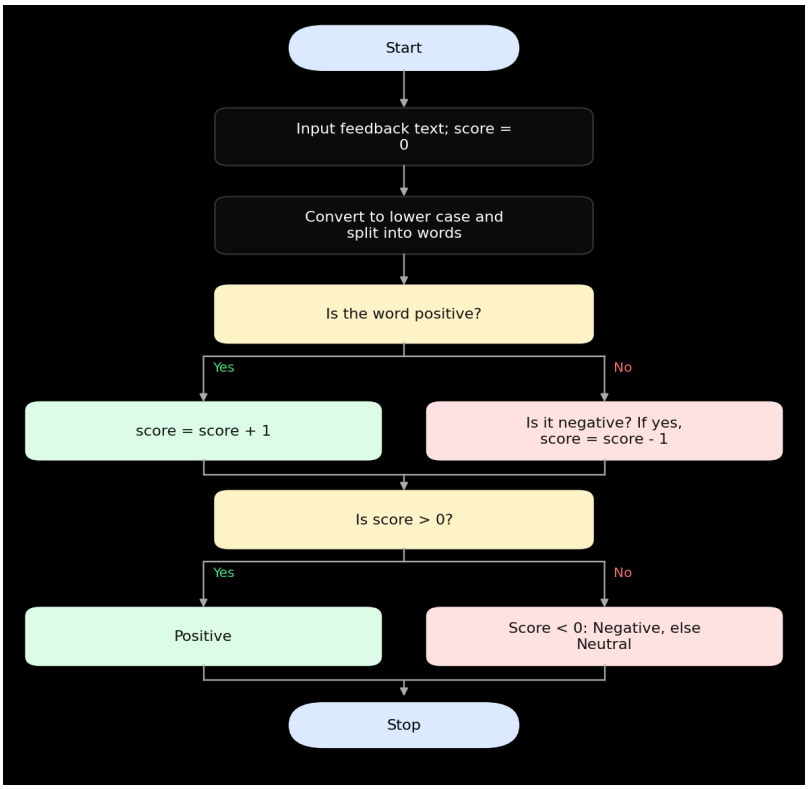
3. Assumptions

1. Positive words: good, excellent, helpful, nice.
2. Negative words: bad, poor, difficult, worst.
3. Each positive word adds 1 to the score and each negative word subtracts 1.
4. A score above 0 is Positive, below 0 is Negative, and exactly 0 is Neutral.
5. Words are separated by spaces.
6. Words that are in neither list are ignored.

4. Algorithm

1. Start the program.
2. Read the feedback text.
3. Define the positive and negative word lists and set score to 0.
4. Convert the text to lower case and split it into words.
5. For each word, add 1 if it is positive or subtract 1 if it is negative.
6. If score > 0, return "Positive".
7. If score < 0, return "Negative".
8. Otherwise, return "Neutral".
9. Stop the program.

5. Flowchart



6. Pseudocode

```
START

SET positive = [good, excellent, helpful, nice]
SET negative = [bad, poor, difficult, worst]
SET score = 0

INPUT text

FOR each word in lowercase(text):
    IF word in positive
        score = score + 1
    ELSE IF word in negative
        score = score - 1

IF score > 0
    RETURN "Positive"
ELSE IF score < 0
    RETURN "Negative"
ELSE
    RETURN "Neutral"

STOP
```

7. Python Source Code / Program

```
def sentiment(text):
    positive = ["good", "excellent", "helpful", "nice"]
    negative = ["bad", "poor", "difficult", "worst"]
    score = 0
    for word in text.lower().split():
        if word in positive:
            score += 1
        elif word in negative:
            score -= 1
    if score > 0:
        return "Positive"
    elif score < 0:
        return "Negative"
    return "Neutral"
test_cases = [
    "good excellent",
    "bad poor",
    "helpful nice",
    "good bad",
    "difficult worst"
]
for i in range(5):
    print("Test Case", i+1, ":", sentiment(test_cases[i]))
```

8. Test Cases

Test case | Feedback text | Expected result

1 | good excellent | Positive

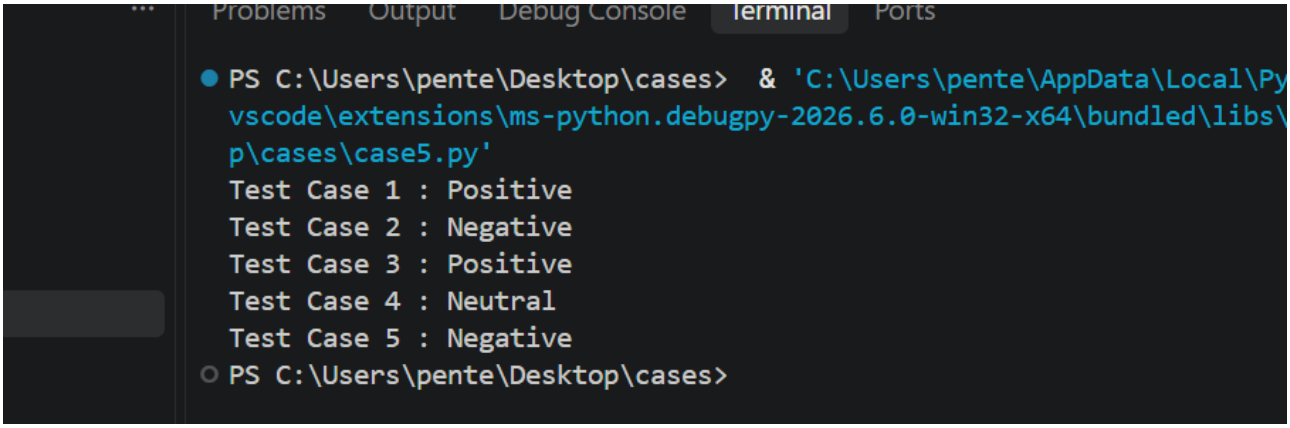
2 | bad poor | Negative

3 | helpful nice | Positive

4 | good bad | Neutral

5 | difficult worst | Negative

9. Output Screenshot



10. Conclusion

This program gives a quick keyword-based sentiment label for student feedback. It is simple and fast, although it does not understand context such as negation.

CASE 06: Text Compression

1. Problem Statement

Strings with many repeated characters can be stored more compactly using run-length encoding. This project develops a Python function that compresses a string by replacing each run of identical characters with the character followed by its count, but returns the original string when compression would not make it shorter.

2. Objectives

1. To compress a string using run-length encoding.
2. To count consecutive repeated characters.
3. To return the original string when the compressed form is not shorter.
4. To handle an empty string safely.
5. To verify the function using five test cases.

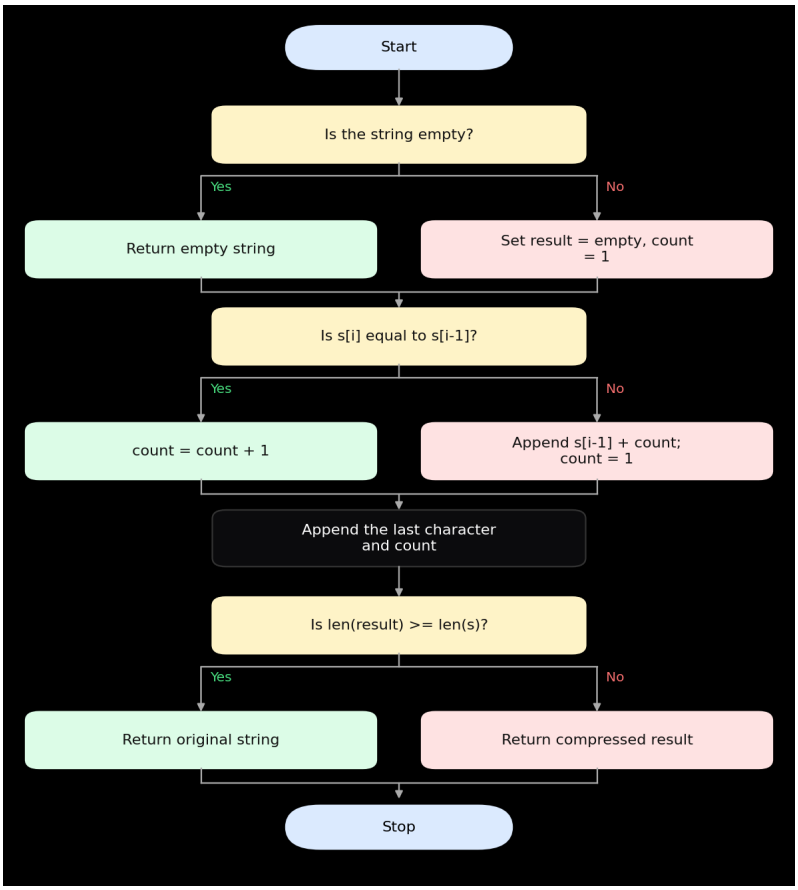
3. Assumptions

1. Only consecutive repeated characters are grouped.
2. Each run is written as the character followed by its count (for example, a3).
3. If the compressed length is greater than or equal to the original length, the original string is returned.
4. An empty string returns an empty string.
5. Counts are written in decimal.

4. Algorithm

1. Start the program.
2. Read the string; if it is empty, return an empty string.
3. Set result to empty and count to 1.
4. Compare each character with the previous one.
5. If they are equal, increase count.
6. Otherwise, append the previous character and count to result and reset count to 1.
7. After the loop, append the last character and its count.
8. If the length of result is not smaller than the original, return the original string.
9. Otherwise, return result.
10. Stop the program.

5. Flowchart



6. Pseudocode

```
START

INPUT s
IF s is empty
    RETURN ""

SET result = ""
SET count = 1

FOR i = 1 TO length(s) - 1:
    IF s[i] = s[i-1]
        count = count + 1
    ELSE
        result = result + s[i-1] + count
        count = 1

result = result + s[last] + count

IF length(result) >= length(s)
    RETURN s
ELSE
    RETURN result

STOP
```

7. Python Source Code / Program

```
def compress(s):
    if not s:
        return ""
    result = ""
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i-1]:
            count += 1
        else:
            result += s[i-1] + str(count)
            count = 1
    result += s[-1] + str(count)
    if len(result) >= len(s):
        return s
    return result
test_cases = [
    "aaabbc",
    "xxxxxy",
    "aabbcc",
    "hhhhhhhh",
    "abcdef"
]
for i in range(5):
    print("Test Case", i+1, ":", compress(test_cases[i]))
```

8. Test Cases

Test case | Input string | Expected result

1 | aaabbc | aaabbc (a3b2c1 is not shorter)

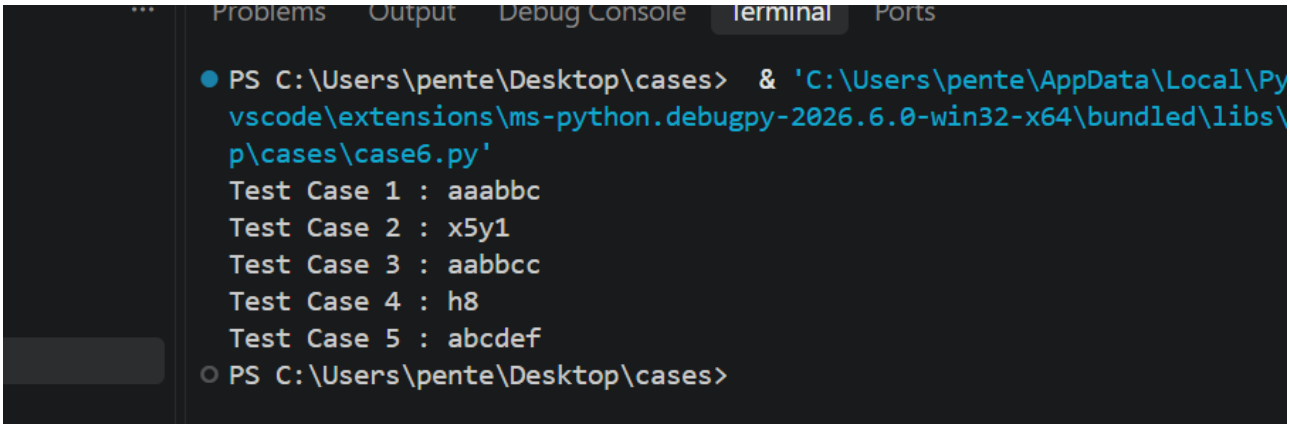
2 | xxxxxy | x5y1

3 | aabbcc | aabbcc (a2b2c2 is not shorter)

4 | hhhhhhhh | h8

5 | abcdef | abcdef

9. Output Screenshot



10. Conclusion

This program compresses repeated characters with run-length encoding and falls back to the original string when compression gives no saving. It therefore never makes a string longer.

CASE 07: Calendar Generator

1. Problem Statement

A monthly calendar must start under the correct weekday and break into a new row after every Sunday. This project develops a Python function that prints a Monday-first calendar for a month, given the weekday on which the month starts and the number of days in the month.

2. Objectives

1. To print a month in a weekly table with weekday headings.
2. To offset the first date according to the starting weekday.
3. To start a new line after every Sunday.
4. To format dates in aligned columns.
5. To verify the function using five different months.

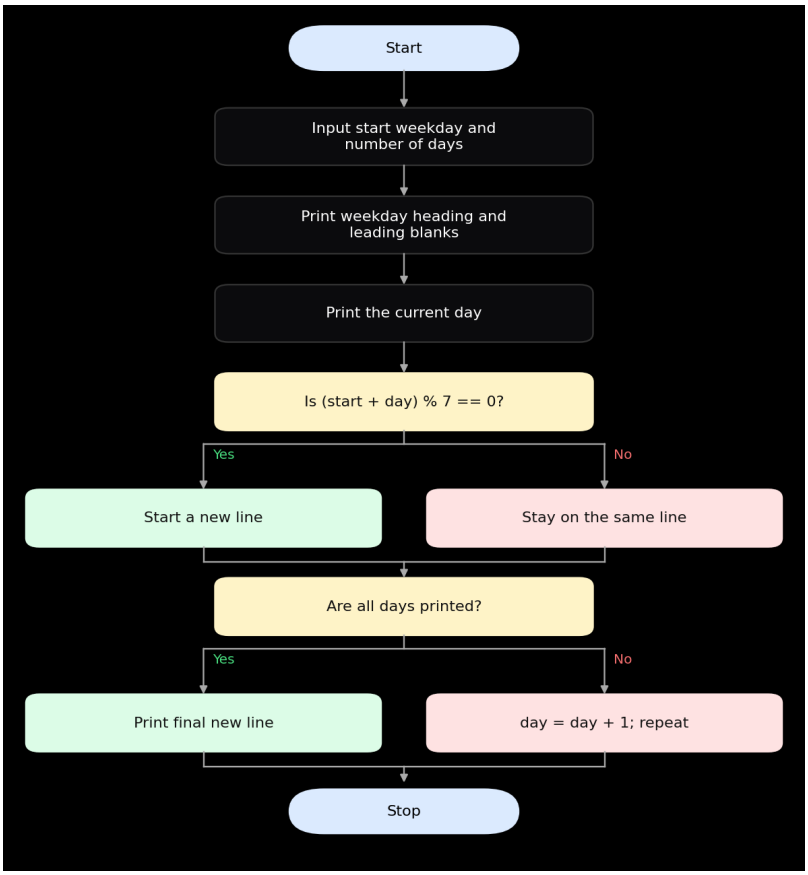
3. Assumptions

1. The week starts on Monday (0 = Mon, 1 = Tue, ... 6 = Sun).
2. The starting weekday is an integer from 0 to 6.
3. The number of days is 28, 29, 30 or 31.
4. Each date is printed in a column three characters wide.
5. Four spaces are printed for every blank day before the 1st.

4. Algorithm

1. Start the program.
2. Read the starting weekday and the number of days.
3. Print the heading Mon Tue Wed Thu Fri Sat Sun.
4. Print blank spaces equal to 4 x start weekday.
5. Set day to 1.
6. Print the day right-aligned in three characters.
7. If (start + day) is divisible by 7, print a new line.
8. Increase day and repeat until all days are printed.
9. Print a final new line.
10. Stop the program.

5. Flowchart



6. Pseudocode

```
START

INPUT start, days

DISPLAY "Mon Tue Wed Thu Fri Sat Sun"
DISPLAY (4 * start) spaces
SET day = 1

FOR i = 1 TO days:
    DISPLAY day (width 3)
    IF (start + day) MOD 7 = 0
        DISPLAY new line
    day = day + 1

DISPLAY new line

STOP
```

7. Python Source Code / Program

```
def calendar(start, days):
    print("Mon Tue Wed Thu Fri Sat Sun")
    day = 1
    print("    " * start, end="")
    for i in range(days):
        print("%3d" % day, end=" ")
        if (start + day) % 7 == 0:
            print()
        day += 1
    print()
test_cases = [
    (0, 28),
    (1, 30),
    (2, 31),
    (3, 30),
    (6, 31)
]
for i in range(5):
    print("Test Case", i+1)
    calendar(test_cases[i][0], test_cases[i][1])
```

8. Test Cases

Test case | (start weekday, days) | Expected layout

1 | (0, 28) | Starts on Monday; four full rows

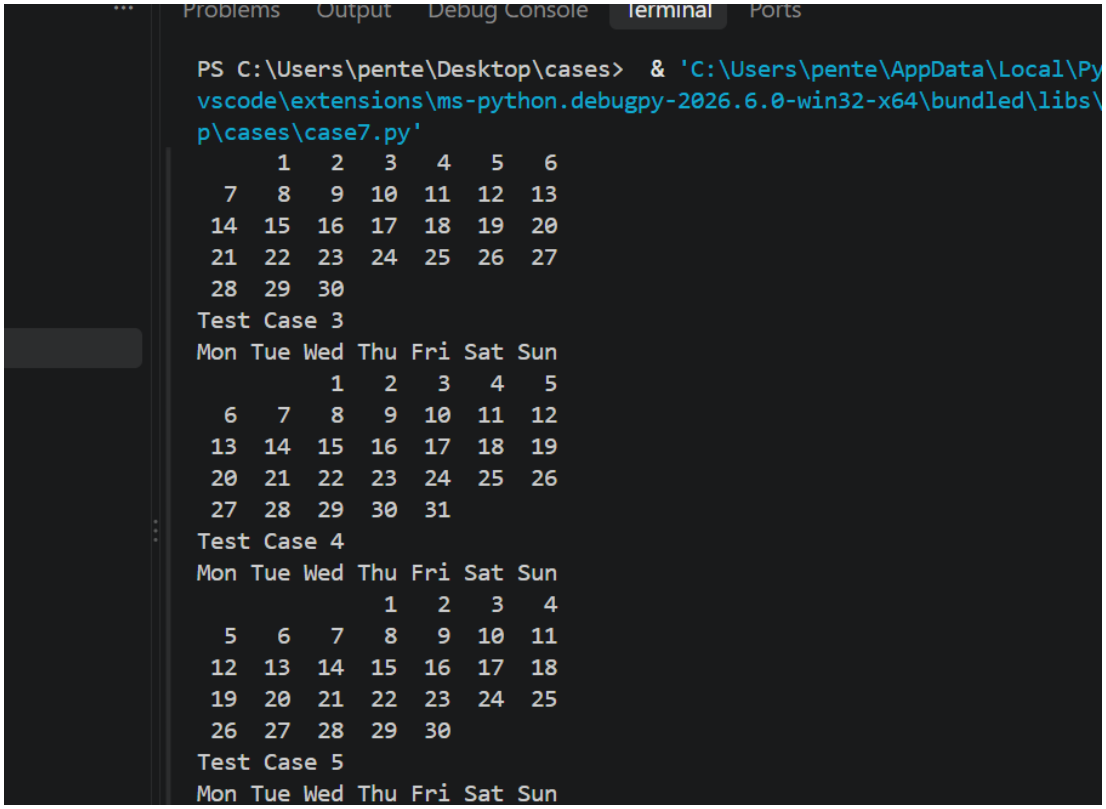
2 | (1, 30) | Starts on Tuesday; 30 days

3 | (2, 31) | Starts on Wednesday; 31 days

4 | (3, 30) | Starts on Thursday; 30 days

5 | (6, 31) | Starts on Sunday; 31 days

9. Output Screenshot



10. Conclusion

This program prints a correctly aligned monthly calendar for any starting weekday. The modulo test makes each week end on Sunday, as the test outputs show.

CASE 08: Examination Eligibility

1. Problem Statement

Before students are allowed to write the final examination, the institution must check their attendance, laboratory marks, internal marks and fee status. This project develops a Python function that returns Eligible only if all four conditions are satisfied.

2. Objectives

1. To check examination eligibility using four conditions.
2. To verify minimum attendance, laboratory marks and internal marks.
3. To confirm that the examination fees are paid.
4. To return a clear Eligible or Not Eligible result.
5. To verify the function using five test cases.

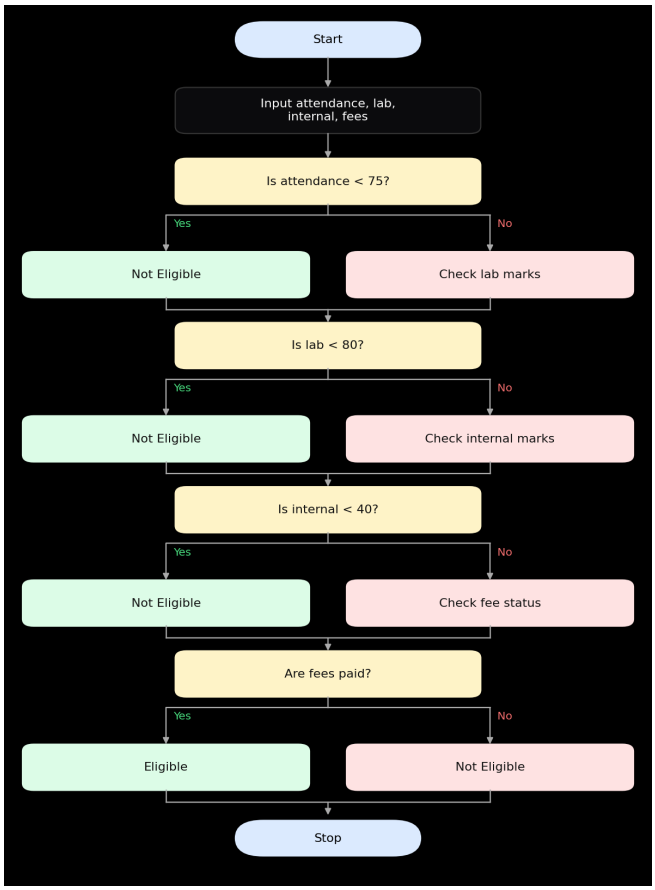
3. Assumptions

1. Minimum attendance is 75%.
2. Minimum laboratory marks are 80.
3. Minimum internal marks are 40.
4. Fee status is entered as "paid" or "unpaid".
5. A student must satisfy all four conditions to be eligible.
6. These limits are sample rules for learning.

4. Algorithm

1. Start the program.
2. Read attendance, lab marks, internal marks and fee status.
3. If attendance is below 75, return "Not Eligible".
4. Else if lab marks are below 80, return "Not Eligible".
5. Else if internal marks are below 40, return "Not Eligible".
6. Else if fees are not "paid", return "Not Eligible".
7. Otherwise, return "Eligible".
8. Stop the program.

5. Flowchart



6. Pseudocode

```
START

INPUT attendance, lab, internal, fees

IF attendance < 75
    RETURN "Not Eligible"
ELSE IF lab < 80
    RETURN "Not Eligible"
ELSE IF internal < 40
    RETURN "Not Eligible"
ELSE IF fees != "paid"
    RETURN "Not Eligible"
ELSE
    RETURN "Eligible"

STOP
```

7. Python Source Code / Program

```
def eligibility(attendance, lab, internal, fees):
    if attendance < 75:
        return "Not Eligible"
    elif lab < 80:
        return "Not Eligible"
    elif internal < 40:
        return "Not Eligible"
    elif fees != "paid":
        return "Not Eligible"
    else:
        return "Eligible"

test_cases = [
    (80, 90, 50, "paid"),
    (60, 90, 50, "paid"),
    (80, 60, 50, "paid"),
    (80, 90, 30, "paid"),
    (80, 90, 50, "unpaid")
]

for i in range(5):
    print("Test Case", i+1, ":", eligibility(*test_cases[i]))
```

8. Test Cases

Test case | (attendance, lab, internal, fees) | Expected result

1 | (80, 90, 50, paid) | Eligible

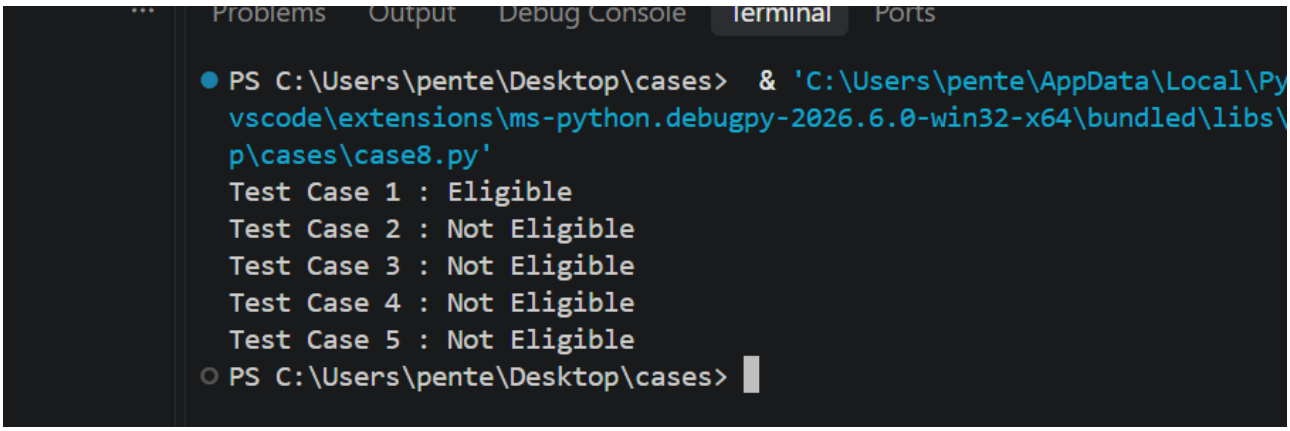
2 | (60, 90, 50, paid) | Not Eligible - attendance

3 | (80, 60, 50, paid) | Not Eligible - lab marks

4 | (80, 90, 30, paid) | Not Eligible - internal marks

5 | (80, 90, 50, unpaid) | Not Eligible - fees

9. Output Screenshot



10. Conclusion

This program decides examination eligibility by checking four conditions in sequence. Each failing condition is shown to produce Not Eligible, while the all-pass case returns Eligible.

CASE 09: Internship Candidate Ranking

1. Problem Statement

Internship selection committees receive many applications and must shortlist candidates fairly. This project develops a Python function that filters candidates by minimum marks, assessment score and attendance, calculates a total score for each eligible candidate, and ranks them from highest to lowest total.

2. Objectives

1. To filter candidates using minimum eligibility criteria.
2. To calculate a total score from marks, assessment and attendance.
3. To rank eligible candidates by total score in descending order.
4. To use the candidate name as a tie-breaker.
5. To verify the function using five test cases.

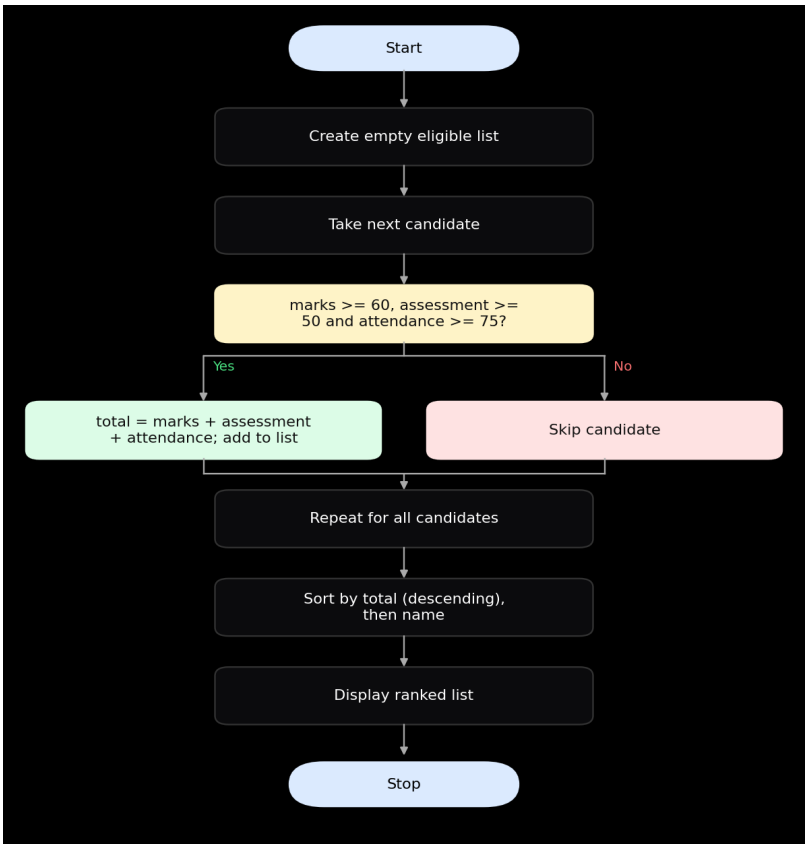
3. Assumptions

1. Each candidate is a (name, marks, assessment, attendance) tuple.
2. Minimum marks are 60, minimum assessment score is 50, and minimum attendance is 75.
3. Total score = marks + assessment + attendance.
4. Candidates failing any condition are not shortlisted.
5. If totals are equal, the name in alphabetical order ranks first.
6. The result is a list of (name, total) pairs.

4. Algorithm

1. Start the program.
2. Create an empty eligible list.
3. For each candidate, read name, marks, assessment and attendance.
4. Check marks >= 60, assessment >= 50 and attendance >= 75.
5. If all are satisfied, calculate the total and add (name, total) to the list.
6. Sort the list by total in descending order, then by name.
7. Return the ranked list.
8. Stop the program.

5. Flowchart



6. Pseudocode

```
START

CREATE empty list eligible

FOR each (name, marks, assessment, attendance):
    IF marks >= 60 AND assessment >= 50 AND attendance >= 75
        total = marks + assessment + attendance
        ADD (name, total) to eligible

SORT eligible by total (descending), then name

RETURN eligible

STOP
```

7. Python Source Code / Program

```
def ranking(candidates):
    eligible = []

    for name, marks, assessment, attendance in candidates:
        if marks >= 60 and assessment >= 50 and attendance >= 75:
            total = marks + assessment + attendance
            eligible.append((name, total))

    eligible.sort(key=lambda x: (-x[1], x[0]))
    return eligible

test_cases = [
    [("A", 80, 70, 90)],
    [("B", 55, 80, 90)],
    [("C", 90, 85, 95)],
    [("D", 70, 60, 80), ("E", 85, 75, 90)],
    [("F", 80, 40, 90)]
]

for i in range(5):
    print("Test Case", i+1, ":", ranking(test_cases[i]))
```

8. Test Cases

Test case | Candidates | Expected result

1 | A (80, 70, 90) | [(A, 240)]

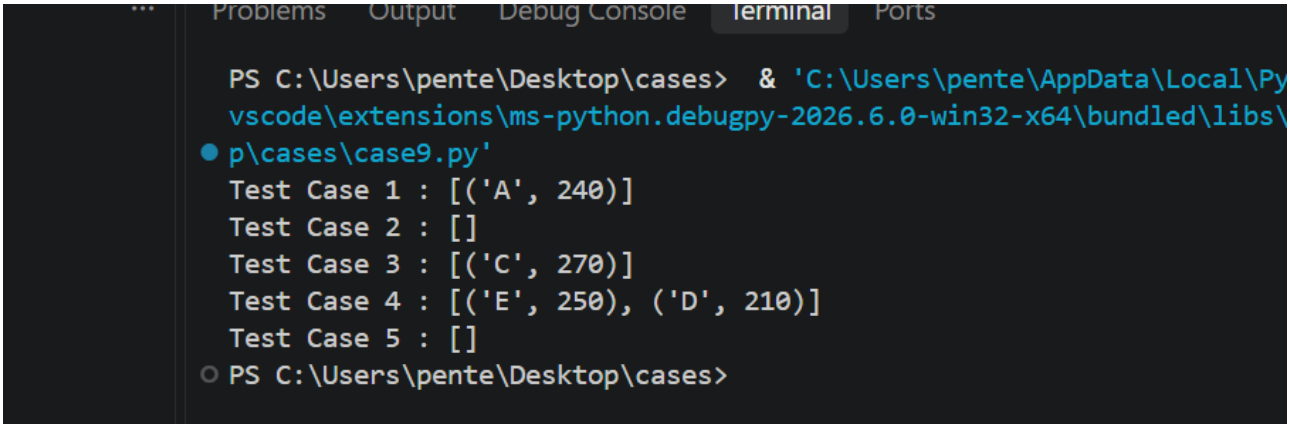
2 | B (55, 80, 90) | [] - marks below 60

3 | C (90, 85, 95) | [(C, 270)]

4 | D (70, 60, 80), E (85, 75, 90) | [(E, 250), (D, 210)]

5 | F (80, 40, 90) | [] - assessment below 50

9. Output Screenshot



10. Conclusion

This program shortlists internship candidates by eligibility and ranks them by total score. The results show that ineligible candidates are removed and that higher totals rank first.

CASE 10: Environmental Monitoring Analyzer

1. Problem Statement

Environmental monitoring stations record repeated readings, such as pollution index values, and officials need a quick summary of air quality. This project develops a Python function that takes a list of readings, calculates the average, maximum and minimum, and classifies the overall condition as Good, Moderate or Poor.

2. Objectives

1. To calculate the average of a list of environmental readings.
2. To find the maximum and minimum readings.
3. To classify the average as Good, Moderate or Poor.
4. To round the average to two decimal places.
5. To verify the function using five test cases.

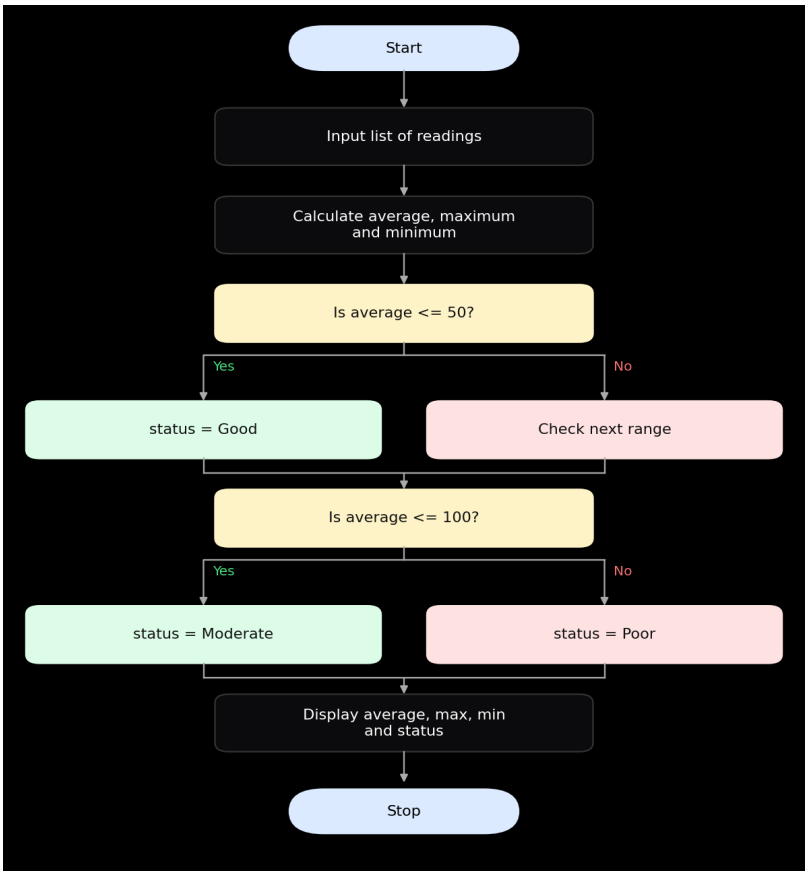
3. Assumptions

1. Each test case is a non-empty list of numeric readings.
2. Average up to 50 is Good.
3. Average above 50 and up to 100 is Moderate.
4. Average above 100 is Poor.
5. The classification is based on the average, not on a single reading.
6. These limits are sample values for learning.

4. Algorithm

1. Start the program.
2. Read the list of readings.
3. Calculate the average = sum of readings / number of readings.
4. Find the maximum and minimum readings.
5. If the average is 50 or less, set status to "Good".
6. Else if the average is 100 or less, set status to "Moderate".
7. Otherwise, set status to "Poor".
8. Return the rounded average, maximum, minimum and status.
9. Stop the program.

5. Flowchart



6. Pseudocode

```
START

INPUT readings

average = SUM(readings) / COUNT(readings)

IF average <= 50
    status = "Good"
ELSE IF average <= 100
    status = "Moderate"
ELSE
    status = "Poor"

RETURN ROUND(average, 2), MAX(readings), MIN(readings), status

STOP
```

7. Python Source Code / Program

```
def analyze(readings):
    average = sum(readings) / len(readings)

    if average <= 50:
        status = "Good"
    elif average <= 100:
        status = "Moderate"
    else:
        status = "Poor"

    return round(average, 2), max(readings), min(readings), status

test_cases = [
    [20, 30, 40],
    [40, 50, 60],
    [60, 80, 100],
    [120, 140, 160],
    [10, 20, 30]
]

for i in range(5):
    print("Test Case", i+1, ":", analyze(test_cases[i]))
```

8. Test Cases

Test case | Readings | Expected (average, max, min, status)

1 | 20, 30, 40 | (30.0, 40, 20, Good)

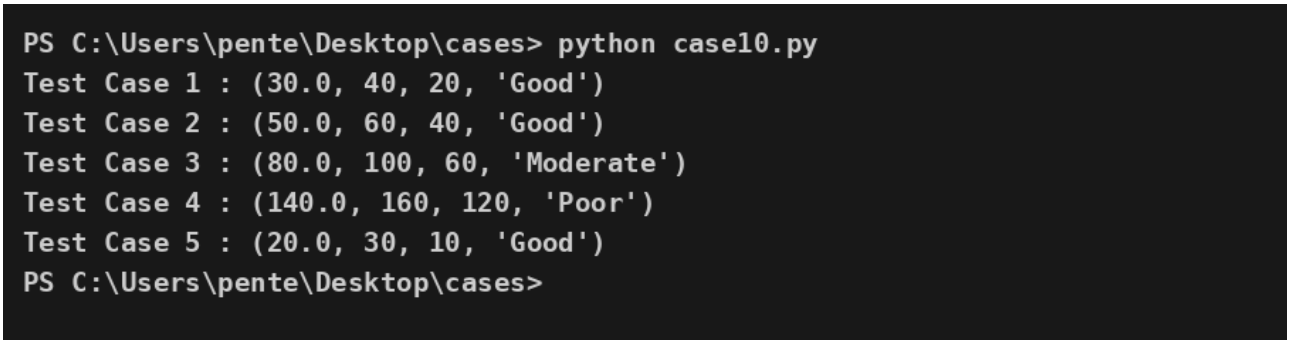
2 | 40, 50, 60 | (50.0, 60, 40, Good)

3 | 60, 80, 100 | (80.0, 100, 60, Moderate)

4 | 120, 140, 160 | (140.0, 160, 120, Poor)

5 | 10, 20, 30 | (20.0, 30, 10, Good)

9. Output Screenshot



10. Conclusion

This program summarises environmental readings and classifies air quality from the average. The boundary case with an average of exactly 50 is correctly classified as Good.